



SDEV 2451

Full Stack Development

Introduction to Full Stack Development - 4

A LEADING POLYTECHNIC COMMITTED TO YOUR SUCCESS

Expectations - What I expect from you

- No Late Assignments
- No Cheating
- Be a good classmate
- Don't waste your time
- Show up to class

Agenda

On the right is what we will cover today.

- Install Dependencies and Configure Tailwind + DaisyUI
- Create List Components
- Create a Controlled Form
- Create Pages
- Set Up Routing with React Router
- Challenge — useEffect and Detail Page
- Key Takeaways

Install Dependencies and Configure Tailwind + DaisyUI

Add routing and styling to a bare Vite + React project before writing any components.

```
npm install react-router-dom tailwindcss @tailwindcss/vite daisyui
```

```
// vite.config.js
import tailwindcss from '@tailwindcss/vite'

export default defineConfig({
  plugins: [react(), tailwindcss()],
})
```

```
/* src/index.css - replace all existing content */
@import "tailwindcss";
@plugin "daisyui";
```

Install Dependencies and Configure Tailwind + DaisyUI (continued)

- `@tailwindcss/vite` runs Tailwind's CSS processing at build time — no separate `tailwind.config.js` needed with Tailwind v4
- `@import "tailwindcss"` injects all of Tailwind's utility classes into the stylesheet
- `@plugin "daisyui"` activates DaisyUI's component layer — classes like `btn`, `table`, `card`, and `badge` with no extra JSX imports
- The old `index.css` must be replaced entirely; leaving default Vite styles causes conflicts with Tailwind's reset

Create List Components

List components accept an array as a prop and render a DaisyUI table — no state, no side effects, just data in and JSX out.

```
function PostList({ posts }) {  
  return (  
    <div className="overflow-x-auto">  
      <table className="table table-zebra w-full">  
        <thead>  
          <tr><th>Title</th><th>Author</th><th>Published</th></tr>  
        </thead>  
        <tbody>  
          {posts.map((post) => (  
            <tr key={post.id}>  
              <td>{post.title}</td>  
              <td>{post.author}</td>  
              <td>{new Date(post.published_at).toLocaleString()}</td>  
            </tr>  
          ))}  
        </tbody>  
      </table>  
    </div>  
  )  
}
```

Create List Components (continued)

- `{ posts }` in the function signature is **destructuring props** — equivalent to `function PostList(props) then const posts = props.posts`
- `table table-zebra` is DaisyUI; `table-zebra` adds alternating row colours with no extra CSS
- `key={post.id}` is required by React when rendering lists — always use a stable unique value like a database ID, **never** the array index
- `??` (nullish coalescing) is useful to render a badge or fallback when a field is `null` or `undefined`

Create a Controlled Form

A controlled form stores every input's value in React state and updates it on every keystroke — the component is always the source of truth.

```
const EMPTY_FORM = { title: '', category: '', body: '' }

function PostForm({ categories, onSubmit }) {
  const [form, setForm] = useState(EMPTY_FORM)

  function handleChange(e) {
    setForm({ ...form, [e.target.name]: e.target.value })
  }

  function handleSubmit(e) {
    e.preventDefault()
    onSubmit(form)
    setForm(EMPTY_FORM)
  }

  return (
    <form onSubmit={handleSubmit}>
      <input name="title" value={form.title} onChange={handleChange} required />
      <button type="submit" className="btn btn-primary">Publish</button>
    </form>
  )
}
```



Create a Controlled Form (continued)

- `useState(EMPTY_FORM)` stores all fields as one object — resetting is a single `setForm(EMPTY_FORM)` call after submit
- `{ ...form, [e.target.name]: e.target.value }` is a **spread update** — `[e.target.name]` matches the input's `name` attribute, so one `handleChange` handles every field dynamically
- `e.preventDefault()` stops the browser from performing a full page reload on submit — without it, all React state is lost
- `onSubmit` is a **callback prop** — the parent page decides what to do with the data; the form stays reusable and unaware of the backend

Create Pages

Pages are thin components that import data and pass it down as props to display components.

```
// src/pages/PostsPage.jsx
import PostList from '../components/PostList'
import { POSTS } from '../mockData'

function PostsPage() {
  return (
    <div>
      <h2 className="text-xl font-semibold mb-3">Posts</h2>
      <PostList posts={POSTS} />
    </div>
  )
}

// src/pages/CreatePostPage.jsx
function CreatePostPage() {
  const navigate = useNavigate()
  function handleSubmit(formData) {
    console.log('Submitted:', formData) // replace with fetch() later
    navigate('/')
  }
  return <PostForm onSubmit={handleSubmit} />
}
```

Create Pages (continued)

- Pages pass `POSTS` directly as the `posts` prop — this is the fundamental **props pattern**: data flows downward from parent to child
- The child component renders whatever it receives and has no knowledge of where the data came from
- Using mock data with field names identical to the real API means the only change needed to go live is swapping the import for a `fetch()` call
- `useNavigate()` returns a `navigate` function — calling `navigate('/')` redirects programmatically after the form submits

Set Up Routing with React Router

`BrowserRouter` wraps the whole app, `Routes` + `Route` map URLs to pages, and `NavLink` highlights the active route.

```
// src/App.jsx
import { BrowserRouter, Routes, Route, NavLink } from 'react-router-dom'

const queryClient = new QueryClient()

function App() {
  return (
    <BrowserRouter>
      <nav className="navbar bg-base-100 shadow px-6">
        <NavLink to="/" end
          className={({ isActive }) =>
            `btn btn-sm ${isActive ? 'btn-primary' : 'btn-ghost'}`>
          Posts
        </NavLink>
      </nav>
      <Routes>
        <Route path="/" element={<PostsPage />} />
        <Route path="/posts/new" element={<CreatePostPage />} />
      </Routes>
    </BrowserRouter>
  )
}
```



Set Up Routing with React Router (continued)

- `BrowserRouter` uses the browser's History API for navigation without full page reloads — must wrap the entire app
- `Routes` renders only the **first matching** `Route` — without it, all matching routes would render at the same time
- `Route path="/" element={<PostsPage />}` maps a URL to a page; `element` takes a JSX element, not a component reference
- `NavLink` receives `isActive` in its `className` callback to toggle between `btn-primary` (active) and `btn-ghost` (inactive)
- The `end` prop on `/` prevents it matching every URL — without it, the home link appears active on every page

Challenge — useEffect and Detail Page

1. Add a useEffect data-fetching layer to PostsPage

- Add `useState` for `posts` (init `[]`) and `loading` (init `true`)
- Add a `useEffect` that calls `mockFetch(POSTS)`, sets `posts` on resolve, sets `loading` to `false` in `.finally()`
- While loading, render a DaisyUI `<span className="loading loading-spinner loading-lg">` in place of the list
- Verify the spinner appears briefly before data loads, then the list renders correctly

2. Add a post detail page

- Create `src/pages/PostDetailPage.jsx` that reads a post `id` from the URL using `useParams` from React Router
- Find the matching post in `POSTS` by `post.id === Number(id)` and display its fields in a DaisyUI `card`

Key Takeaways

- **Components and props** — small focused functions that accept data from their parent via props and return JSX; data always flows downward
- **Destructuring props** — `function PostList({ posts })` makes the component's contract explicit at a glance
- **Controlled form inputs** — every input's value lives in `useState` ; one `handleChange` with `[e.target.name]` handles all fields
- **useState reset pattern** — storing the whole form as one object makes resetting trivial:
`setForm(EMPTY_FORM)` after submit
- **Callback props** — pass `onSubmit` as a prop so the form stays reusable and unaware of the backend
- **React Router** — `BrowserRouter` provides context, `Routes` + `Route` map URLs, `NavLink` highlights the active link, `useNavigate` redirects programmatically

WE ARE ESSENTIAL
TO ALBERTA





Example

Let's go do an example together

A LEADING POLYTECHNIC COMMITTED TO YOUR SUCCESS